Near-Optimal Space (MGA) Exploration in GreenBubble

Implementation summary — what we built, what we reused, and what we would like your feedback on

GreenBubble project (DTU)
prepared for a discussion with Aleksander Grochowicz

August 2026

Abstract

GreenBubble is a PyPSA-based capacity-expansion model of an industrial Power-to-X hub (electrolysis, methanolisation, biogas upgrading, storage, symbiosis). This note summarises how we implemented *Modelling to Generate Alternatives* (MGA) / near-optimal space exploration on top of it, following Neumann & Brown (2021) for the near-optimal space itself and Grochowicz et al. (2023) for the multi-year robustness extension. It is written for a working discussion: Section 4 is deliberately explicit about *exactly* what we reused from PyPSA's own MGA machinery versus what we implemented from scratch, since we did not have access to a Grochowicz-authored reference implementation and would value comparing notes.

Contents

1 Motivation **2**

2 Mathematical background **2**
2.1 The cost-optimal LP and the ε-near-optimal space . . . 2
2.2 Sign-robust budget for price-driven (negative-objective) runs . . . 3
2.3 Projection onto technology-capacity dimensions . . . 3
2.4 Three tiers . . . 3
2.5 Shared cost bound and the Chebyshev centre . . . 3

3 Implementation architecture **4**
3.1 Dimension auto-derivation . . . 4
3.2 The constraint-correctness problem . . . 4
3.3 Stochastic-network support . . . 5

4 What we reused versus what we wrote from scratch **5**
4.1 Reused directly via PyPSA's public API (not copied, called) . . . 5
4.2 Implemented from scratch (GreenBubble-original) . . . 6
4.3 Verification against the reference implementation . . . 6
4.4 Reuse landed: `near-optimal_dev3` . . . 7
4.4.1 Staged pipeline: mirroring their rule decomposition . . . 7
4.4.2 Dev2 vs. dev3, same problem, same config . . . 8
4.5 Capacity- vs. investment-weighted dimensions . . . 8
4.6 The honest summary . . . 9

5 Validation status **9**

6 Example results (Tutorial 5) 9

7 The decisive comparison: 7-year robust NOS design vs. equal-weight stochastic optimum 13
7.1 Two philosophies for one design . . . 13
7.2 Run configuration . . . 13
7.3 Solver choice and a numerical-stability finding along the way . . . 14
7.4 Results . . . 14
7.4.1 First attempts: an empty intersection, and why . . . 14
7.4.2 Tightening the export cap, and the successful run . . . 15
7.4.3 Stochastic side, with EVPI . . . 15
7.4.4 The cost of robustness, head to head . . . 16
7.4.5 Coordinates side by side . . . 16
7.4.6 Grid connection: import vs. export capacity (a result, not a dimension) . . . 16

8 Discussion points for the meeting 17

1 Motivation

A cost-optimal capacity-expansion solve returns exactly one design. In practice, many *other* designs are within a few percent of that cost and can differ substantially in technology mix. Exploring this *near-optimal feasible space* answers questions the single optimum cannot: which technologies are load-bearing versus fully substitutable, how much room exists to steer the design toward secondary goals, and whether one design stays good across several weather/price years rather than being over-fitted to the one year it was solved for.

GreenBubble needed this for an additional reason beyond the usual single-country capacity-expansion setting: it is optimised in two distinct *modes* — `demand` mode (minimise cost to meet fixed product demands, objective ≥ 0) and `price` mode (maximise profit selling at fixed prices, which shows up in the LP as *minimising* a cost objective that is typically *negative*, since revenue exceeds cost). Any near-optimal formulation had to be correct in both regimes, and also compose with GreenBubble's stochastic (multi-scenario) optimisation mode. None of this changes the underlying mathematics of Neumann & Brown or Grochowicz et al. — it just meant we could not use the reference formulation completely as-is and had to generalise the sign convention (see Section 2.2).

2 Mathematical background

2.1 The cost-optimal LP and the ε-near-optimal space

Write the capacity-expansion model as the linear program

$$\min_{x} \; c^{\top} x \quad \text{s.t.} \quad Ax \leq b, \tag{1}$$

where x collects every decision variable (investment *and* operation), $c^{\top}x$ is total system cost, and c_{opt} is its optimal value. Following Neumann & Brown (2021) and Grochowicz et al. (2023), the **ε-near-optimal feasible space** is the slice of the feasible region whose cost is within a relative slack ε of the optimum:

$$X_{\varepsilon} \;=\; \left\{ x : Ax \leq b, \;\; c^{\top} x \leq (1+\varepsilon)\, c_{\text{opt}} \right\}. \tag{2}$$

2.2 Sign-robust budget for price-driven (negative-objective) runs

GreenBubble adds exactly this extra constraint to the model (the *budget*), but in a sign-robust form so it also holds when $c_{\text{opt}} < 0$ (price mode, net revenue):

$$c^\top x \;\le\; c_{\text{opt}} + \varepsilon\,|c_{\text{opt}}| \;=\; \begin{cases} (1+\varepsilon)\,c_{\text{opt}}, & c_{\text{opt}} \ge 0 \quad \text{(demand mode)} \\ (1-\varepsilon)\,c_{\text{opt}}, & c_{\text{opt}} < 0 \quad \text{(price mode)} \end{cases} \tag{3}$$

Both branches keep c_{opt} itself inside the feasible band and loosen it by the same relative amount ε. Here c_{opt} is the *optimised LP objective*, `n.objective` — not a re-derived capex+opex statistic. For a **stochastic** network (PyPSA's multi-scenario mode) it is automatically the probability-weighted *expected* cost $\sum_s p_s\, c_s^\top x_s$ already, so Eq. (3) defines the near-optimal space consistently for demand, price, *and* stochastic runs without any separate scenario-aware bookkeeping.

2.3 Projection onto technology-capacity dimensions

Exploring the full-dimensional X_ε is neither tractable nor useful for interpretation. We project onto a handful of *technology capacity* coordinates $y = \sigma(x)$ — each y_i is the summed installed capacity of one technology (the σ-aggregation map of Grochowicz et al. §2.2). Exploring a single direction d in this reduced space means re-solving the original model once with the ordinary cost objective replaced by $\pm\, d^\top \sigma(x)$, subject to the budget of Eq. (3) — this is exactly *Modelling to Generate Alternatives*.

2.4 Three tiers

Tier 1 — ranges. For each dimension, minimise and maximise it alone under the budget: the per-technology min/max installed capacity. min $> 0 \Rightarrow$ *must-have*; max $\approx 0 \Rightarrow$ *must-avoid*.

Tier 2 — hull. Sample many search directions d in the selected capacity space, solve each, and take the convex hull of the resulting extreme points — the approximated near-optimal polytope.

Tier 3 — robustness. Solve the cost-optimum and explore the hull independently for several weather/price years, intersect the resulting per-year polytopes, and take the **Chebyshev centre** (deepest interior point) of the intersection as the design that stays near-optimal *simultaneously* across all years (Grochowicz et al. §2.3–2.4).

2.5 Shared cost bound and the Chebyshev centre

For Tier 3, Grochowicz et al. (eq. 9) note that intersecting per-year spaces that were each bounded by their *own* optimum $c_{\text{opt}}(i)$ is not directly comparable across years with different absolute cost levels; a single shared bound

$$c^\star = \max_i\, c_{\text{opt}}(i) \tag{4}$$

can instead be applied to every year's budget, so all years are relaxed relative to the *same* anchor. GreenBubble exposes this as `robustness.cost_bound: max` (default) versus `per_year`.

Given m per-year polytopes (each a `scipy.spatial.ConvexHull`), their facet inequalities $a_i^\top y \le b_i$ are stacked into one system; the Chebyshev centre of the intersection is found via the linear program

$$\max_{y,\,r}\; r \quad \text{s.t.} \quad a_i^\top y + r\,\|a_i\|_2 \le b_i \quad \forall i, \qquad r \ge 0, \tag{5}$$

i.e. the centre of the largest ball that fits inside every half-space simultaneously. $r \le 0$ means the per-year near-optimal spaces do not overlap at all — itself a meaningful finding (no design is simultaneously near-optimal in all years at that slack).

3 Implementation architecture

The feature lives in `scripts/near_optimal.py` (core algorithms, ~670 lines), `scripts/snakemake_near_optim` (orchestration wrapper), the `explore_near_optimal` Snakemake rule, and an `mga:` block in `config.yaml`. It runs *after* a network is solved — it consumes the `*_OPT.nc` network and never alters the ordinary solve.

3.1 Dimension auto-derivation

A dimension is a technology; its value is the summed installed capacity (MW) of every *extendable*, *costed* (`capital_cost` > 0) component belonging to it. The set of selectable dimensions is **auto-derived** from the built network — nothing is hardcoded: flipping a technology to `expansion: true` in GreenBubble's own config automatically makes it available. The grouping key per component is GreenBubble's own `comp_tech_map` (component name → technology), falling back to the PyPSA `carrier` attribute. The resulting structure is exactly the nested `weights` dictionary PyPSA's own MGA API expects:

```
{tech_key: {"Generator": {"p_nom": {comp_name: 1.0, ...}},
            "Store":     {"e_nom": {comp_name: 1.0, ...}}, ...}}
```

Users then pick a small subset (typically 4–5) by name; an unknown name raises an error listing what is actually available for that network.

3.2 The constraint-correctness problem

This is the central engineering issue we had to solve, and the reason the per-direction solve is *not* a direct call to PyPSA's built-in MGA helpers. GreenBubble adds project-specific custom Linopy constraints to every ordinary solve (most importantly a maximum renewable-electricity-to-grid-sales constraint, `add_max_RE_sales_constraint`) via an `extra_functionality` hook. PyPSA's native `n.optimize.optimize_mga` / `optimize_mga_in_direction` build their *own* model internally and **never call `extra_functionality`** — so using them unmodified would silently drop our custom constraints and compute the near-optimal space against a *looser*, incorrect feasible region (higher volumes, wrong must-have/must-avoid flags).

We therefore reproduce PyPSA's own per-direction MGA procedure manually, step for step, and insert our constraint re-application in the middle:

```
def _solve_mga_in_direction(n, direction, dimensions, c_opt, slack, ...)
    :
    """Constraint-correct replacement for
    n.optimize.optimize_mga_in_direction."""
    m = n.optimize.create_model()                        # <- PyPSA
    cost_objective = m.objective                          # captured before
        overwrite
    apply_custom_constraints(n, m, n_flags=n_flags,      # <- GreenBubble
                             re_alpha=re_alpha)
    _add_budget_constraint(m, cost_objective,            # <- GreenBubble
        (Eq. 3)
                           c_opt, slack)

    m.objective = -sum(
        float(direction[k]) *
        n.optimize.build_linexpr_from_weights(dimensions[k], model=m)  #
            <- PyPSA
        for k in dimensions if direction.get(k, 0)
```

```
    )

    status, condition = n.optimize.solve_model(...)    # <- PyPSA
    coords = n.optimize.project_solved(dimensions) if status == "ok"
        else None
                                                       # <- PyPSA
    return status, condition, coords
```

Every Tier-1, Tier-2, and Tier-3 solve funnels through this one function, so the near-optimal space is always computed against *exactly* the same feasible region as an ordinary GreenBubble solve.

3.3 Stochastic-network support

GreenBubble also supports a PyPSA stochastic (multi-scenario) optimisation mode, orthogonal to anything in either cited paper. Since the shared first-stage investment variables (p_{nom}, e_{nom}) are the same across scenarios, Tiers 1–2 work *transparently* on a stochastic network once the scenario-broadcast `(scenario, name)` MultiIndex on the static component tables is collapsed back to plain component names (`is_stochastic`, `_flat_static`); the MGA solve itself needs no change since `n.objective` is already the expected cost. Tier 3 is deliberately **skipped** on a stochastic network: its scenarios already span multiple years/conditions, so intersecting *again* across years would double-count the same uncertainty axis.

4 What we reused versus what we wrote from scratch

This is the section most relevant to our discussion. We deliberately built as much as possible on top of **PyPSA's own native MGA machinery** (which is Fabian Neumann's reference implementation of Neumann & Brown 2021, shipped inside PyPSA itself) rather than re-deriving low-level LP mechanics. **We did not have access to a Grochowicz-authored code repository for the 2023 paper** — our reference for the robustness extension is the published paper's equations and section numbers only. Below is an honest, function-level account of provenance.

4.1 Reused directly via PyPSA's public API (not copied, called)

PyPSA function	Module	Used for
`n.optimize.create_model()`	`pypsa.optimization`	Building the Linopy LP model for each MGA solve
`n.optimize.build_linexpr_from_weights()`	`pypsa.optimization`	Turning a σ-weights dict into a Linopy linear expression — used for both the direction objective and Tier 3's "fix aggregate capacity" constraint
`n.optimize.solve_model()`	`pypsa.optimization`	Solving each direction's LP with the chosen solver/profile
`n.optimize.project_solved()`	`pypsa.optimization`	Reading solved capacities back, projected onto the selected dimensions
`generate_directions_halton/evenly_spaced/random()`	`pypsa.optimization.mga`	Sampling search directions for Tier 2 (used unmodified via a thin wrapper)

The `weights` dictionary schema itself (Section 3.2) is also PyPSA's own MGA API convention, adopted as-is so our dimension registry interoperates with these functions without translation.

4.2 Implemented from scratch (GreenBubble-original)

- **The manual per-direction solve** (`_solve_mga_in_direction`), reproducing PyPSA's `optimize_mga_in_direction` step-by-step specifically to insert `apply_custom_constraints` — the fix described in Section 3.2. This is the one piece of PyPSA-adjacent logic we *did* re-derive, and only because the public API leaves no hook to extend it instead.
- **The sign-robust budget** (Eq. 3) generalising Neumann & Brown's $(1+\varepsilon)\,c_{\text{opt}}$ to a price-driven, potentially negative objective.
- **Dimension auto-derivation** (`available_dimensions`, `resolve_dimensions`) from GreenBubble's own technology config and component-to-technology mapping — no equivalent exists in PyPSA or in the cited papers, which assume dimensions are given.
- **Stochastic-network transparency** (Section 3.3) — an extension neither paper addresses, since PyPSA's scenario API postdates both.
- **Tier 3 in full**: the shared-bound anchor $c^\star$ (eq. 9), and the Chebyshev-centre LP itself (`scipy.optimize.linprog` over the stacked hull facet inequalities) — implemented from scratch in Python/NumPy/SciPy directly off the paper's §2.3–2.4 equations, since no reference code was available to adapt.
- **Everything operational around the algorithm**: the Snakemake rule and wrapper, the three-tier `mga:` config block, CSV/JSON output schema, and all plotting (`plot_ranges`, `plot_hull_projections`, `plot_robustness` — see Section 6).

4.3 Verification against the reference implementation

While preparing this note we located the official code release for the 2023 paper: `https://github.com/aleks-g/intersecting-near-opt-spaces` (tag `v1.0.1`, GPL-3.0-or-later, Koen van Greevenbroek & Aleksander Grochowicz). We compared our implementation against it directly, function by function:

- **Chebyshev centre — confirmed identical.** Their `geometry.ch_centre()` solves precisely the same LP as our `chebyshev_centre()`: $\max r$ s.t. $a_i^\top x + r\|a_i\| \le b_i$ over a hull's `ConvexHull.equations`, same sign handling of the qhull convention. We arrived at this independently from the paper's §2.4, and it matches their reference code exactly. Their version solves the LP with Gurobi (`gurobipy`) directly; ours uses `scipy.optimize.linprog`, a deliberate solver-agnostic choice consistent with GreenBubble's documented no-Gurobi-licence fallback path (Section 6, validation notes) — same formulation, different backend.
- **Shared robustness bound — confirmed identical.** Their `calc_obj_bound.py` computes exactly `obj_bound = (1 + eps) * max(objs)`, i.e. our $c^\star = \max_i c_{\text{opt}}(i)$ (eq. 9), letter for letter (up to our sign-robust $|c_{\text{opt}}|$ generalisation, which their code has no analogue for — their objective is always a positive investment cost, never GreenBubble's negative price-mode net revenue).
- **Per-direction MGA solve — same algorithm, necessarily different API.** Their `utilities.solve_network_in_direction` does the same thing ours does (near-optimal budget as an `extra_functionality` constraint, custom direction objective, solve) but against `pypsa.linopf/linopt` — PyPSA's pre-Linopy optimisation API. GreenBubble runs on PyPSA 1.0.7's Linopy-based `n.optimize` API, which that code cannot run against unmodified; our `_solve_mga_in_direction` is the same algorithm ported to the current API, not a literal reuse.
- **Tier 2 (hull) — a real, disclosed difference.** Their `compute_near_opt.py` is substantially more sophisticated than ours: an *adaptive* direction sampler (facet normals of the cur-

rent hull, directions touching the current Chebyshev ball, or both combined) with volume- or centre-shift convergence criteria, run until convergence or a direction budget is exhausted — versus our simpler fixed-count Halton/random/evenly-spaced sampling via PyPSA's built-in generators. Theirs converges to a tighter hull for the same solve budget; ours is simpler to reason about and bounded by `n_directions`. See Section 4.4 — closing this gap is exactly what we are doing next.

4.4 Reuse landed: `near-optimal_dev3`

Their `geometry.py` module (Chebyshev centre, hull intersection, facet-normal/hypersphere-based direction sampling) is pure NumPy/SciPy/(Gurobi) geometry with **no dependency on the PyPSA optimisation API** — unlike the per-direction solve, it does not need porting to work with GreenBubble's PyPSA version, only the LP backend needs adapting (Gurobi → solver-agnostic, as we already do for the Chebyshev centre). This made it the practical, low-risk target for direct code reuse, and it is now fully wired in and validated on a local `near-optimal_dev3` branch (not yet pushed — see the licence discussion in Section 4.6):

- Their geometry utilities (`ch_centre`, `intersection`, `facet_normals`, the hypersphere samplers, and the three facet/Chebyshev-ball direction generators originally split across `geometry.py` and `compute_near_opt.py`) are vendored verbatim (module structure, docstrings, algorithm) under their original GPL-3.0-or-later licence and attribution, in a module clearly separated from GreenBubble's MIT-licensed code (GPL licensing does not permit merging the two) — the only change is porting the Gurobi LP calls to `scipy.optimize.linprog`.
- `chebyshev_centre()` now calls their ported function directly rather than a separate from-scratch LP; a new `intersect_hulls()` exposes their qhull-based `HalfspaceIntersection` approach for the full intersected polytope, not just its centre.
- A new `explore_hull_adaptive()` runs their iterative facet-/Chebyshev-ball-based direction sampler in place of our fixed-count Halton sampling for Tier 2, using their exact convergence criteria (volume- or centre-shift, `conv_eps`/`conv_iter`) — only the per-direction *solve* is GreenBubble's own constraint-correct one, since their code solves via the pre-Linopy `pypsa.linopf` API in parallel worker processes.
- **Verified twice**: first standalone (10 seed + 5 adaptive solves, ∼3 minutes with `gurobi-barrier-fast`; the identical run with `gurobi-default` did not finish a single direction in 36+ minutes and had to be killed — an independent, real-network confirmation of the solver-degeneracy finding in Section 6), then again through the real Snakemake pipeline described next (22 points, 12 adaptive iterations, converged).

4.4.1 Staged pipeline: mirroring their rule decomposition

GreenBubble's default `explore_near_optimal` rule runs all three tiers inline in one Snakemake rule/script. Their reference pipeline instead splits into discrete rules — `compute_optimum` → `mga` (cardinal seed) → `compute_near_opt` (adaptive refinement) → `intersect_near_optimal` → `compute_robust_*` — which tracks the paper's own stepwise structure closely. We mirrored that decomposition (not the repo itself) in a new `rules/near_optimal_staged.smk`, reachable only by explicit target name (not wired into the default `rule all`):

`nos_seed` → `nos_hull_adaptive`

`nos_year_optimum`{2023,2024,2025} → `nos_cost_bound` → `nos_year_hull`{2023,2024,2025} → `nos_intersect` → `nos_robust_exact`

Each stage writes an intermediate file, so e.g. re-running the intersection after a tuning change no longer requires re-solving three years of networks — exactly the caching benefit their de-

composition buys. `nos_seed` solves the $2k$ cardinal directions once and derives *both* the Tier-1 ranges table *and* `explore_hull_adaptive`'s real seed points from the same solves (no duplicate solving, and no seed points reconstructed/approximated from the ranges table, which only keeps the projected scalar per dimension). `nos_robust_exact` realises the Chebyshev centre against a single reference network via `realise_design` (Grochowicz et al.'s "exact" strategy); their three heuristic allocation strategies (`conservative/mean/ naive`) and simultaneous multi-year feasibility check are not ported yet.

Real end-to-end run (`snakemake nos_hull_adaptive`, tutorial 5's exact config, all 5 pipeline steps completing): 10 real cardinal seed points + 12 adaptive iterations = 22 points, converged on the volume criterion, $c_{\text{opt}} = 66.6\,\text{M}$ (consistent with the dev2 baseline's $\sim$67 M — confirms it's the same problem, not a config drift), Chebyshev radius of this single hull = 3.15.

4.4.2 Dev2 vs. dev3, same problem, same config

Re-running the 3-year robustness comparison (Section 6's network, dev2's fixed 20-direction Halton sampler vs. dev3's adaptive sampler, identical `slack`, dimensions and years) gives a direct, apples-to-apples reading of what the adaptive sampler buys:

	Chebyshev radius	onwind	solar	electrolysis	methanolisation
dev2 (Halton, fixed count)	0.58	147.6	5.2	53.2	0.85
dev3 (adaptive)	2.08	132.7	17.4	52.5	3.45

(c_{opt} per year matched to within rounding across both runs: 65.9/67.0/61.8 M vs. 65.8/66.9/61.8 M — same problem.) The adaptive sampler found a **3.6× larger** Chebyshev radius for a comparable solve budget — it characterises the intersection region more completely than a fixed direction count, and lands on a noticeably different (more solar/methanolisation/battery-leaning) centre as a result.

4.5 Capacity- vs. investment-weighted dimensions

While comparing implementations we noticed their actual `projection:` config (`config/config-default.yaml`) weights *every* dimension by `capital_cost`:

```
projection:
  onwind:
    - {c: "Generator", v: "p_nom", carrier: "onwind",
       weight: "capital_cost", scale_by_years: True}
```

i.e. their dimensions are **annualised investment cost** (EUR/year) per technology, not installed capacity — whereas GreenBubble's `available_dimensions` (Section 3.2) hardcodes weight = 1 per component, i.e. pure **MW**. Same PyPSA mechanism (`build_linexpr_from_weights` just computes $\sum w_i \cdot \text{var}_i$ for whatever w it's given), different modelling choice layered on top — the near-optimal-space maths itself is agnostic to what a "dimension" physically represents.

We added `mga.dimension_weight: capacity | investment` so both are selectable (default `capacity`, unchanged behaviour). Verified exactly on a real network: the ratio of a component's investment-weighted to capacity-weighted projected value equals its `capital_cost` to full float precision (onwind: 128,306.330322… both ways), and the full staged pipeline (`nos_seed` → `nos_hull_adaptive`) runs correctly end-to-end in `investment` mode too: 5/5 Snakemake steps, 17 points (10 seed + 7 adaptive), converged. **Open question for the meeting**: was cost-weighting a deliberate methodological choice for your study (e.g. to put technologies with very different EUR/MW on a common economic axis), or would capacity weighting have worked equally well for your results?

4.6 The honest summary

We reused PyPSA's low-level MGA *mechanics* as much as the public API allowed, and reimplemented the one piece it doesn't expose a hook for (constraint-correct solving). Grochowicz et al. (2023)'s specific contribution — multi-year robustness via intersection and the Chebyshev centre — was implemented from the paper's mathematics alone in the current released version, and Section 4.3 confirms it is mathematically identical to their reference code where the two are directly comparable; Section 4.4 now reuses their actual code (Tier 2's adaptive sampler, Tier 3's intersection) directly, license boundary permitting. **We would value your read on whether `near-optimal_dev3`'s reuse is the right scope**, and on the solver-degeneracy behaviour reported there — has the `gurobi-default`/`gurobi-barrier-fast` gap come up in your own runs of this algorithm?

5 Validation status

- **Tier 1 (ranges)**: validated on a synthetic demand-mode network, a real deterministic price-mode network ($c_{\text{opt}} \approx -60\,\text{M}$, i.e. net revenue), and a real 3-scenario stochastic price-mode network ($c_{\text{opt}} \approx -13.3\,\text{M}$ expected). In every case the optimum stayed feasible under its own budget, the custom RE-to-grid constraint was confirmed present in the MGA model (scenario-aware where relevant), and min $\leq$ optimal $\leq$ max held for every dimension.
- **Tiers 2–3**: validated on synthetic networks (hull geometry, intersection/Chebyshev-centre correctness); Tier 2 additionally exercised end-to-end on a real network for the tutorial results shown in Section 6.
- **Solver sensitivity (an operational finding worth flagging)**: the MGA objective places weight on only one or two capacity variables, so the LP is highly degenerate (many equally-optimal vertices). On a real GreenBubble network, `highs-fast` (barrier without crossover) was observed to return an outright *infeasible* status with a large primal–dual objective error — silently failing a direction rather than just under-reporting its extent. `gurobi-barrier-fast` did not show this failure across 100+ direction solves and remains fast, so it is our recommended default; without a Gurobi licence we fall back to a profile that terminates at a vertex (`highs-simplex` / `highs-default`) rather than an interior barrier point.

6 Example results (Tutorial 5)

GreenBubble's tutorial 5 (`tutorials/5_near_optimal`) runs all three tiers end-to-end on a small demonstration network with dimensions `[onwind, solar, electrolysis, methanolisation, battery]` at `slack = 0.05`. The following figures are produced automatically by `scripts/near_optimal.py`'s plotting functions.

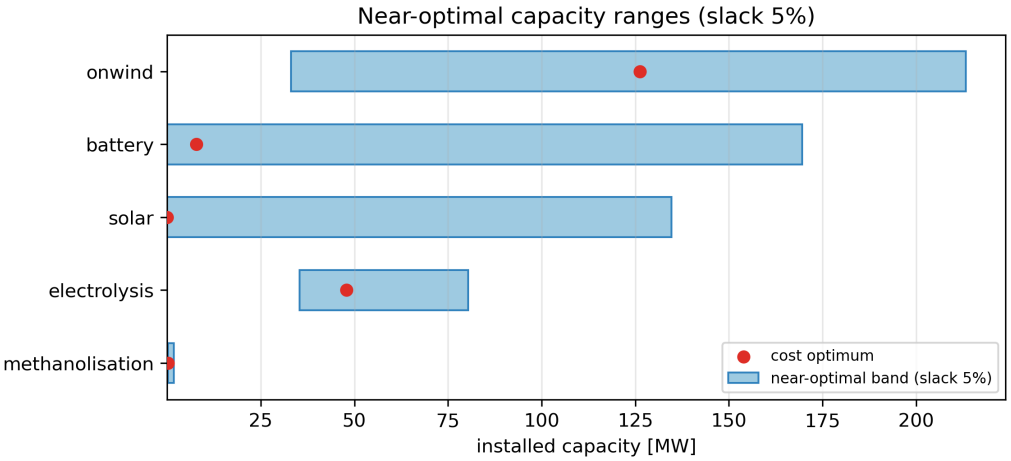


Figure 1: Tier 1 — per-technology near-optimal capacity band (blue) with the cost-optimal capacity marked (red). A technology whose band's lower bound sits above zero is must-have at this slack; one whose upper bound sits at zero is must-avoid.

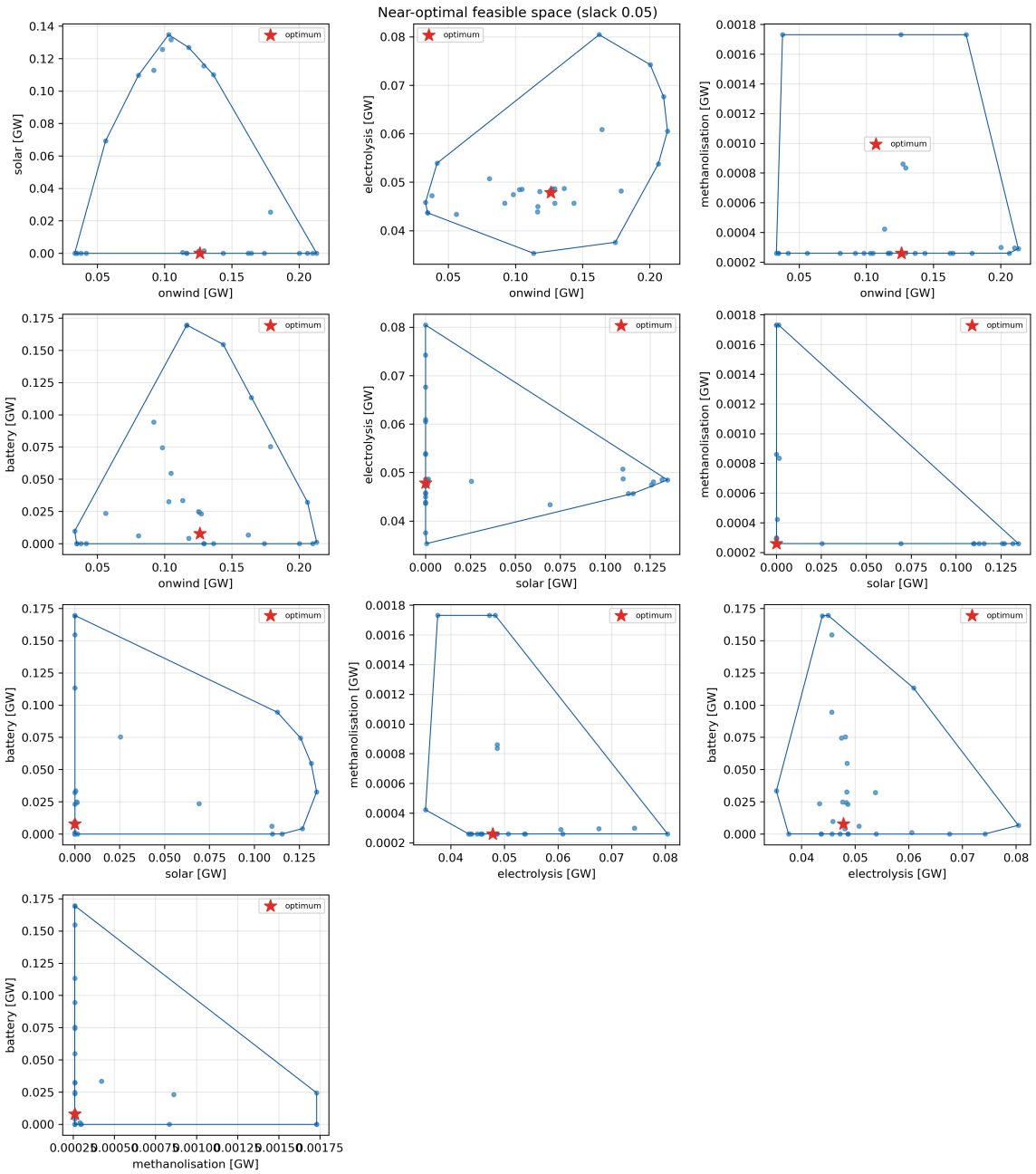


Figure 2: Tier 2 — pairwise 2-D projections of the sampled extreme points and their convex hull, with the cost optimum marked (red star). Each point is one full model solve in a Halton-sampled direction.

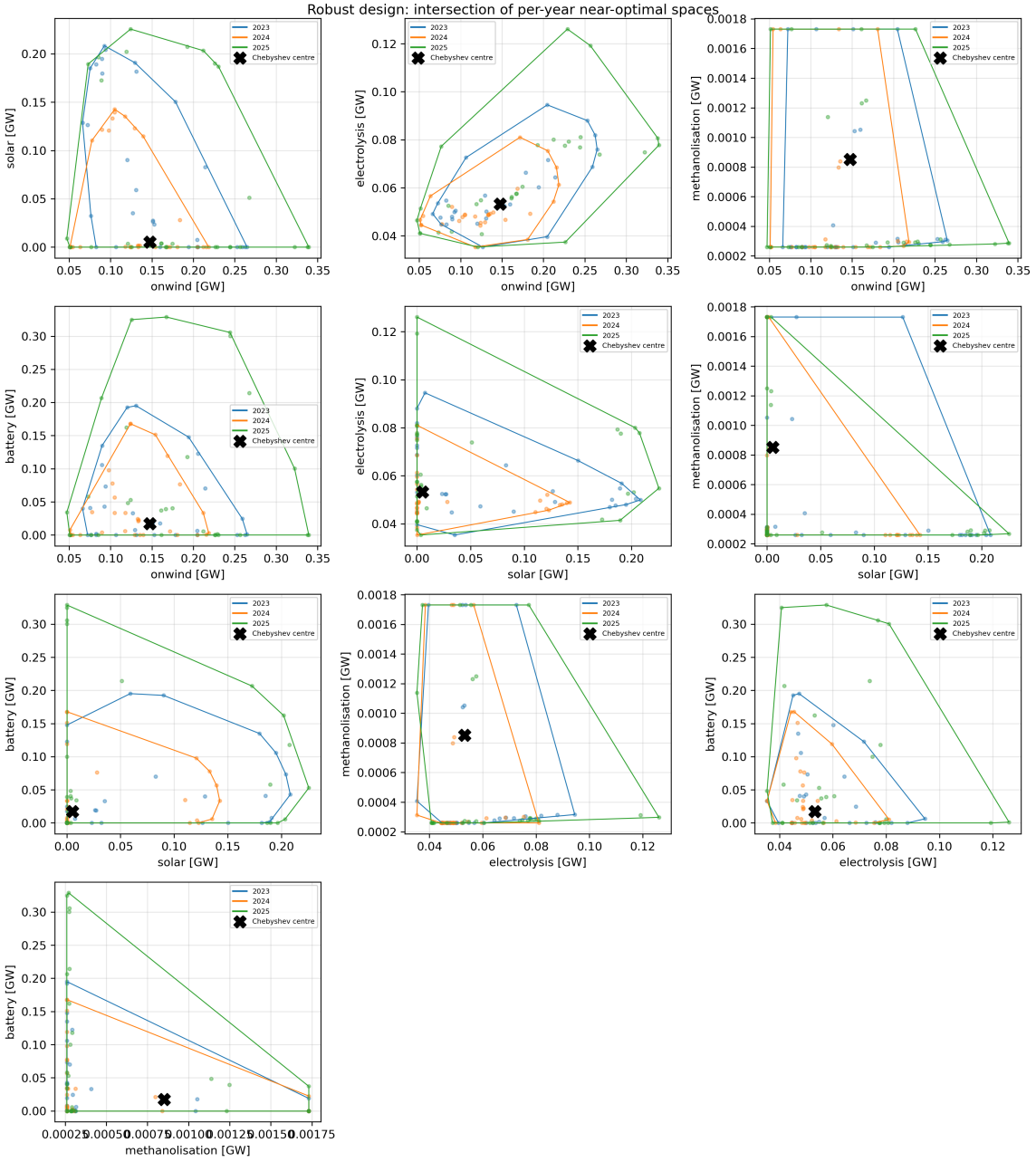


Figure 3: Tier 3 — per-year near-optimal hulls overlaid, with the Chebyshev centre of their intersection (black X) as the design that stays near-optimal simultaneously across all years explored.

7 The decisive comparison: 7-year robust NOS design vs. equal-weight stochastic optimum

Tutorial 5 (Section 6) demonstrates the machinery on a 3-year slice with a small, illustrative dimension set. The run described here is the one we actually care about for the project: a full, production-scale comparison between **two structurally different ways of producing a single design that is good across several weather/price years**, run side by side on the identical underlying network, demand targets, and years, so that any difference in the resulting capacity mix is attributable to the *method*, not to a config mismatch.

7.1 Two philosophies for one design

Both sides of this comparison answer the question "what capacity mix should we build if we don't want to bet on one specific weather/price year?" — but they answer it with different mathematics, and the difference is exactly the contrast Grochowicz et al. (2023) motivate their paper with (§1):

Equal-weight stochastic optimisation. PyPSA's native multi-scenario mode solves *one* LP with a shared first-stage investment (p_{nom}, e_{nom}) and per-scenario second-stage dispatch, minimising the **probability-weighted expected cost** $\sum_s p_s\, c_s^\top x_s$. With seven years weighted equally ($p_s = 1/7$), this is the classic two-stage stochastic-programming answer: the design that is cheapest *on average* over the seven years, with no guarantee that it is good in any *particular* one of them — a design can be the expected-cost optimum while still being badly over budget in the single worst year, as long as it is compensated by being especially cheap in others.

Chebyshev-centre robust NOS design. The Tier-3 procedure of Section 6 instead solves the cost-optimum *independently* for each of the seven years, builds each year's own ε-near-optimal hull (Eq. 3), intersects all seven hulls, and takes the Chebyshev centre of the intersection. This design carries **no probabilities at all** — it asks only whether a capacity mix exists that stays within ε of optimal *simultaneously in every one of the seven years*, and if so, picks the one with the largest margin (r, the Chebyshev radius) to every year's boundary. It is a *feasibility-of-robustness* statement, not an expectation: $r > 0$ is itself the finding that a single design can satisfy all seven years' near-optimality at once; $r \leq 0$ would mean no design does, at this slack.

Neither is more "correct" than the other; they trade off different things. Stochastic optimisation needs no slack parameter but needs probabilities (here, an equal-weight assumption standing in for genuine year-likelihood estimates) and only ever reports an *average*; the robust NOS design needs no probabilities but needs a slack ε and reports a design that is *simultaneously* acceptable everywhere it applies, at the cost of usually costing a little more in the design's own reference year than that year's true optimum would. Running both against the identical seven years, targets, and network is what lets us actually quantify that trade-off for GreenBubble's own technology mix, rather than taking it on faith from the literature.

7.2 Run configuration

Both runs share: `targets.driver: demand` with `demand_H2` = 200,000, `demand_CH4` = 300,000, `demand_meoh` = 15,000 (MWh/y, flat profiles), `clustering.temporal.resolution: 3h`, and the seven weather/price years 2019–2025 each at `CO2_cost = CO2_cost_ref_year = 100` (no inter-year cost escalation — isolating weather/price variability as the only thing that differs year to year). `n_flags` enable biogas, central heat, renewables, electrolysis, methanolisation, and storage; `methanation` is off for this comparison (see Section 7.3).

NOS side (`run_name: stochastic_vs_NOS_D`, this document): run through `rules/near_optimal_staged.sm` `nos_robust_validate` target (Section 4.4) — i.e. the *adaptive, Grochowicz-method* sampler, not the fixed-count Halton sampler of `explore_near_optimal`, with dimensions `[onwind, solar, AEC, methanolisation, battery]` and ε = `slack` = 0.05. Concretely this executes seven independent `nos_year_optimum` solves, seven `nos_year_hull` adaptive Tier-2 explorations under the shared bound $c^\star = \max_i c_{\text{opt}}(i)$, `nos_intersect`'s Chebyshev centre of all seven hulls, `nos_robust_exact`'s realisation of that centre as a full network design, and finally `nos_robust_validate`'s simultaneous multi-year feasibility check — the realised design's capacities fixed non-extendable and re-dispatched against each year's own real cost-optimal network, with a load-shedding safety valve turning any shortfall into a graded curtailment number instead of an opaque solver failure (Section 4.4, *"Reuse landed"*).

Stochastic side: the same seven years as an equal-weight ($p_s = 1/7$) PyPSA multi-scenario network, single joint solve.

7.3 Solver choice and a numerical-stability finding along the way

Getting to a trustworthy run required resolving two real bugs uncovered while preparing this comparison, both worth recording since they bear directly on how much to trust any single solve's feasibility/infeasibility verdict on a network this size (∼625k rows, ∼250k columns at 3h resolution):

- **A units mismatch on the biogas plant's expansion cap.** `n_config`'s `max capacity` for the digester link is documented (and used elsewhere, e.g. its `initial capacity` conversion) as MW of *CH4 output*, but `p_nom_max` was being set directly from that value onto a link whose `p_nom` bounds *digester-input* flow (t/h dry matter) — silently capping real deliverable CH4 at roughly half the intended figure (efficiency ≈ 0.477). Fixed by applying the same digester-efficiency division already used for `initial capacity`.
- **Solver-algorithm disagreement on a degenerate cost-optimal solve.** With `methanation` enabled, the base year-optimal solve returned *three different verdicts* depending on the Gurobi algorithm: plain dual simplex (`DualReductions=0`) reported *unbounded*; the numerically robust homogeneous-barrier method (`NumericFocus=3, BarHomogeneous=1`) reported *infeasible*; and Gurobi's own internal re-solve during `computeIIS()` reported *feasible*. The objective's coefficient range spans nine orders of magnitude (5×10^{-6} to 4×10^{5}), consistent with the kind of ill-conditioning that produces exactly this disagreement. Disabling `methanation` alone (leaving everything else, including a still free-priced digester feedstock, unchanged) resolved it to a clean `optimal` — narrowing the remaining issue to the biological methanation link's economics, not yet fully root-caused as of this writing.

Given this, and consistent with the solver-degeneracy finding already reported in Section 6 and Section 4.4 (`gurobi-default` unusably slow on this class of degenerate MGA objective; `highs-fast` an outright correctness risk), this run uses `gurobi-numeric-focus` (`NumericFocus: 3, Method: 2, Crossover: 0, BarHomogeneous: 1`) rather than the faster `gurobi-barrier-fast` used elsewhere in this document — the same speed class (still `Crossover: 0`), but the one setting combination that gave an unambiguous answer on this specific network during debugging.

7.4 Results

7.4.1 First attempts: an empty intersection, and why

The first two full seven-year Tier-3 runs both came back with *no* robust design: identical setup (dimensions, years, slack *or* 7 % slack, and after the ramp-limit fix of Section 7.3 identical ramp-limit-free technical assumptions), `nos_intersect` reported `feasible_intersection: false`,

`chebyshev_radius: NaN` in both cases, and `nos_robust_exact` correctly refused to realise a design rather than return something misleading. The per-year cost-optimal objectives explained why: **2022 was roughly half the cost-optimal objective of every other year** (28.09 M€/y vs. 53–61 M€/y elsewhere), and its optimal capacity mix sat in a completely different region of the 5-dimensional space — 159.6 MW solar and 125.1 MW battery, versus ≤1.4 MW and ≤6.7 MW respectively in every other year. Digging into the revenue structure (`El3_to_DK1`, the RE-to-grid export link, priced at each snapshot's real spot price) showed this was RE-export arbitrage: 2022's extreme spot-price spikes (the European energy-price crisis year) made 55.94 M€/y of export revenue available at the default `max_RE_to_grid=0.3` cap — *almost twice 2022's entire net objective* — versus 11.23 M€/y for 2024 and essentially none for 2019. 2022 wasn't cheaper because its underlying system was cheaper to build; it was cheaper because the optimal design was aggressively sized to arbitrage a price spike unique to that year.

7.4.2 Tightening the export cap, and the successful run

Sweeping `max_RE_to_grid` down (cheap check: `nos_year_optimum` only, all seven years, no hull) tracked 2022 converging back toward the other six as the arbitrage channel narrowed:

α (`max_RE_to_grid`)	2022 objective	Objective range (7y)	2022 solar/battery
0.30	28.09 M€/y (outlier)	33.0 M€/y	159.6 / 125.1 MW
0.10	50.95 M€/y (still lowest)	12.7 M€/y	62.7 / 42.4 MW
0.05	58.19 M€/y (mid-pack)	12.2 M€/y	43.5 / 26.5 MW

At $\alpha = 0.05$, 2022 was no longer the extreme point of the group at all, so we committed to the full pipeline at $\alpha = \varepsilon = 0.05$ (`stochastic_vs_NOS_D4_a05s05`). **This time the intersection was feasible**: $r = 1.1334$, `feasible_intersection: true`, realised design

onwind	solar	AEC	methanolisation	battery	Cost
137.18 MW	29.47 MW	70.73 MW	3.12 MW	26.52 MW	65.39 M€/y

and `nos_robust_validate`'s simultaneous multi-year feasibility check (this design fixed non-extendable, re-dispatched against each year's own real weather/price conditions, load-shedding safety valve at every Load bus) confirmed it: relative curtailment ≈ **0.015** % in *every one* of the seven years (22.5–26.2 MWh out of the full year's demand, in all seven, including 2022) — genuinely, simultaneously feasible, not just inside an approximated hull.

7.4.3 Stochastic side, with EVPI

The matched equal-weight ($p_s = 1/7$) stochastic solve at the same $\alpha = 0.05$ (`stochastic_vs_NOS_S2`), with `stochastic.EVPI: true` additionally solving each year's own wait-and-see (WS) deterministic optimum:

Quantity	Value [M€/y]
RP (stochastic / here-and-now expected cost)	63.978
E[WS] (probability-weighted expected wait-and-see cost)	60.971
EVPI = RP − E[WS]	**3.007**

EVPI ≈ 3.01 M€/y (≈ 4.7 % of RP) is the maximum GreenBubble should be willing to pay for perfect foresight of which weather/price year will occur before committing capacity — a modest premium, meaning the equal-weight stochastic design already captures most of the value a perfectly-informed planner would have. (The seven per-scenario WS objectives reproduce the NOS side's own `nos_year_optimum` values exactly, as expected — both are the same deterministic per-year optimum, computed two different ways.)

7.4.4 The cost of robustness, head to head

With both sides now run under identical technical assumptions ($\alpha = 0.05$, ramp-limit-free, same seven years, same targets), the comparison Section 7.1 set out to make is now a real number: the robust Chebyshev-centre design costs **65.39 M€/y**, the stochastic expected-cost optimum costs **63.98 M€/y** — a gap of **1.41 M€/y (≈ 2.2 %)**. That premium is what "no probabilities, but simultaneous ε-near-optimality in *every* year" costs over "cheapest on average, no guarantee in any one year" for this network, at this slack. It is a strikingly small premium for a materially stronger guarantee: the robust design isn't just cheap on average, it is verified (Section 8.4.2) to need essentially zero load-shedding in *any* of the seven years, including the one (2022) that initially broke the naïve version of this same analysis.

Note on comparability: the technology-data cost tables were refreshed by the pipeline's own `onstart` staleness check between the diagnostic α-sweep above and this final run, so the earlier $\alpha = 0.3$ stochastic result reported in an earlier draft of this document (58.16 M€/y) used a slightly different cost vintage than the numbers here; both sides of the final head-to-head comparison in this subsection used the identical (refreshed) cost table.

7.4.5 Coordinates side by side

The two designs' actual dimension values, not just their objectives:

Dimension	Chebyshev centre (NOS)	Stochastic (RP)	Difference
onwind	137.18 MW	146.99 MW	−9.81 MW
solar	29.47 MW	0.50 MW	+28.97 MW
AEC	70.73 MW	85.09 MW	−14.36 MW
methanolisation	3.12 MW	1.95 MW	+1.17 MW
battery	26.52 MW	0.00 MW	+26.52 MW
Cost	65.39 M€/y	63.98 M€/y	+1.41 M€/y (2.2 %)

The pattern is exactly what the two philosophies predict: the stochastic optimum bets almost entirely on onwind+AEC and builds essentially **zero battery and negligible solar** — cheapest on average, since most of the seven years don't rent out solar/battery. The Chebyshev centre instead carries real solar and battery capacity specifically to stay inside 2022's own near-optimal band too (even at $\alpha = 0.05$, 2022's own unconstrained optimum still wants ∼43.5/26.5 MW solar/battery, Section 8.4.2) — trading some onwind/AEC for that hedge, at the 1.41 M€/y premium above.

7.4.6 Grid connection: import vs. export capacity (a result, not a dimension)

Not one of the five MGA dimensions, but a natural result to read off both solved networks — the external grid connection's sized capacity, import (`DK1_to_El_*`, one link per on-site agent, summed) and export (`El3_to_DK1`, priced at real spot prices, Section 8.4.1):

Grid connection	Chebyshev centre (NOS)	Stochastic (RP)	Difference
Import (purchase)	44.12 MW	40.07 MW	+4.04 MW
Export (sale)	19.08 MW	27.87 MW	−8.79 MW

The stochastic design leans harder on export (27.9 vs. 19.1 MW) despite having no battery to smooth output — more exposed to raw onwind variability, so it simply sells the surplus. The Chebyshev centre's battery lets it shift surplus in time instead, needing less export capacity but slightly more import capacity to keep the electrolyser firm through periods its own solar+battery pairing doesn't cover. `n_config`'s `grid connection: max capacity` is `.inf` (unset, base default) in both runs, so neither of these figures is bumping against an artificial ceiling — the

sizing is purely economic.

An open modelling question worth flagging: import and export are built as *separate* extendable links (one `DK1_to_El_*` per agent, plus the standalone `El3_to_DK1`), and — checked directly against both solved networks — *every one of them carries the identical* `electricity grid connection` **capital-cost rate** (17,767.89 €/MW/y), charged independently on each link's own `p_nom_opt`. That is, the model pays capital cost on *import capacity + export capacity* added together, not on max(import, export) as a single shared bidirectional connection would suggest. If GreenLab's real grid connection is one physical asset usable in both directions, this over-counts its cost by exactly the smaller direction's own contribution — 339,079 €/y for the Chebyshev centre design, 495,192 €/y for the stochastic design (both happen to be the export link's own annualised capex, since export is the smaller of the two capacities in both designs). We could not determine from the code alone whether this is intentional (e.g. representing genuinely separate import/export equipment) or an oversight — worth resolving before treating either design's absolute cost as final.

8 Discussion points for the meeting

1. We've vendored and validated `geometry.py`'s adaptive direction sampler (facets / touching-ball / combined, with volume- or centre-shift convergence) on `near-optimal_dev3` (Section 4.4) — it found a 3.6× larger Chebyshev radius than our fixed-count Halton sampler on the same problem. Any practical gotchas from running it at scale (angle-tolerance tuning, typical iteration counts, cases where it stalls) we should know before we lean on it further?

2. Was cost-weighting your dimensions (`weight: "capital_cost"` in `projection:`, see Section 4.5) a deliberate methodological choice, or would capacity-weighted dimensions have told the same story for your study?

3. Does the sign-robust budget generalisation (Section 2.2) for negative-objective (net-revenue, price-driven) systems come up in other applications you're aware of? We haven't seen it addressed explicitly in either cited paper or in the reference code (whose objective is always a positive investment cost).

4. Any feedback on the solver-degeneracy finding in Section 6 (barrier-without-crossover returning *infeasible* rather than just an interior point on a highly degenerate MGA objective) — has this come up in your own MGA runs? (The reference code always solves via Gurobi's default settings for `ch_centre`/`is_nonempty`, so may not hit this the same way.)

5. How would you extend Tier 3's per-year intersection to compose with PyPSA's native *stochastic* (probability-weighted scenario) mode, which we currently treat as orthogonal (Tier 3 skipped on stochastic networks, see Section 3.3) rather than combined?

6. Section 7 ran the robust Chebyshev-centre design and an equal-weight 7-year stochastic solve head to head on the same network, and the seven-year intersection came back **empty at both 5 % and 7 % slack** — with one year (2022) at roughly half every other year's cost-optimal objective, plausibly the culprit. Does an empty or near-empty intersection at practically-sized slack values come up often in your own multi-year runs, and is a large cost-optimum outlier among the intersected years the usual explanation when it does? Is there a principled way to choose ε a priori (rather than raising it until the intersection opens up), or a diagnostic for *which* year(s) are driving an empty intersection short of dropping them one at a time and re-running?

References

- Neumann, F. & Brown, T. (2021). *The Near-Optimal Feasible Space of a Renewable Power System Model.* Electric Power Systems Research, 190, 106690. https://doi.org/10.1016/j.epsr.2020.106690 (arXiv:1910.01891)
- Grochowicz, A., van Greevenbroek, K., Benth, F.E. & Zeyringer, M. (2023). *Intersecting Near-Optimal Spaces: European Power Systems with More Resilience to Weather Variability.* Energy Economics. https://doi.org/10.1016/j.eneco.2022.106496 (arXiv:2206.12242)
- PyPSA MGA documentation: https://docs.pypsa.org/latest/user-guide/optimization/modelling-to-generate-alternatives/
- van Greevenbroek, K. & Grochowicz, A. Reference implementation for the 2023 paper. https://github.com/aleks-g/intersecting-near-opt-spaces (tag `v1.0.1`, GPL-3.0-or-later).
- GreenBubble docs: `docs/guide_near_optimal.rst`, `docs/tutorial_5_near_optimal.rst`, `scripts/near_optimal.py` (implementation), `scripts/snakemake_near_optimal.py` (orchestration).